PRÀCTICA 01 · AEA1 · RA1

Supervisor de processos i comunicació IPC

Enunciat de laboratori · Java 21 · 8 hores

REPTE
Desenvolupa una aplicació que creï processos externs, els identifiqui, en controli l'entrada i la sortida, gestioni temps d'espera i connecti processos mitjançant pipes.

Mòdul	MP 0490 - Programació de serveis i processos
Durada	8 hores
Alineació	AEA1 · RA1
Tecnologia	Java 21 o Kotlin/JVM
Producte final	Supervisor multiprocés, proves, README i evidències

Situació de partida

Pren com a inspiració el patró Fork-Join: un supervisor divideix el treball, inicia tasques independents i n'agrega els resultats. En aquesta pràctica, però, cada tasca s'ha d'executar com un procés real amb PID propi, no només com un fil.

1. Objectius i requisits

En acabar la pràctica has de ser capaç de:

- Distingir executable, procés, servei i fil.
- Identificar PID, PPID, estat i durada d'un procés.
- Crear processos amb ProcessBuilder sense construir ordres interpretables pel shell.
- Enviar dades per stdin i capturar stdout i stderr sense interbloqueig.
- Gestionar codis de retorn, timeout, cancel·lació i finalització forçada.
- Connectar dos processos amb una pipe productor-consumidor.
- Executar tasques independents amb un límit de concurrència.

Requisits previs

- JDK 21 i IntelliJ IDEA o un editor equivalent.
- Git i terminal.
- Sistema Linux o contenidor de laboratori.
- Coneixements bàsics de Java o Kotlin.

NORMES DE TREBALL
Treballa només en l'entorn autoritzat. No incorporis secrets, tokens ni dades personals. Cada membre ha de poder executar i explicar la solució. Declara l'ús d'IA i verifica qualsevol proposta abans d'integrar-la.

Escenari mínim

El programa ha d'incloure els rols lògics SUPERVISOR, WORKER, PRODUCER i CONSUMER. Cada missatge de diagnosi ha de permetre identificar, com a mínim, el rol, el PID, el PPID i el fil actual.

```
[WORKER-BETA pid=4312 ppid=4280 thread=main] pas 2/3
```

2. Desenvolupament pas a pas

1. Crear el repositori. Afegeix README, .gitignore i una estructura clara per al codi i les proves.
2. Inventariar processos. Executa eines del sistema i registra PID, PPID, estat, durada i ordre.
3. Llançar un procés. Implementa una funció que rebi l'executable i els arguments com una llista.
4. Capturar stdout i stderr. Llegeix els dos fluxos simultàniament i conserva marca temporal i origen.
5. Enviar entrada. Escriu dades a stdin d'un procés interactiu senzill i tanca correctament el flux.
6. Modelar el resultat. Retorna codi de sortida, durada, stdout, stderr i estat de timeout en un objecte immutable.

Ordres d'observació suggerides

```
ps -eo pid,ppid,stat,etime,cmd | grep ProcessForkJoinDemo
pstree -p <PID_DEL_SUPERVISOR>
```

Documenta quines ordres són portables i quines depenen del sistema. L'aplicació no pot dependre de pstree per funcionar.

Mapa inicial de processos

Rol	PID	PPID	Estat observat	Codi retorn
Supervisor				
Worker Alpha				
Worker Beta				
Worker Gamma				

3. Supervisió, concurrència i IPC

1. Afegir timeout. Permet configurar una durada màxima. Prova una finalització ordenada i, si cal, una de forçada.
2. Cancel·lar de manera segura. Evita processos orfes i registra la causa del tancament.
3. Connectar dos processos. Canalitza stdout del productor cap a stdin del consumidor i recull els dos codis de retorn.
4. Executar en paral·lel. Llança tres workers independents amb un màxim de dos processos simultanis i agrega els resultats.
5. Gestionar senyals. En Linux, prova una interrupció controlada i explica finalització ordenada i forçada.
6. Crear proves. Comprova èxit, ordre inexistent, stderr, timeout, stdin, pipe i cancel·lació sense Internet.
7. Documentar la diagnosi. Inclou arbre de processos, un bloqueig evitat i instruccions exactes de reproducció.

Flux IPC obligatori

```
PRODUCTOR --stdout--> SUPERVISOR --stdin--> CONSUMIDOR
                                    |
                                    `--> resultat i diagnosi
```

CONDICIÓ IMPORTANT
No n'hi ha prou amb executar una ordre del shell que ja contingui una pipe. La connexió entre els fluxos l'ha d'implementar el vostre programa.

Resultat esperat de cada execució

Camp	Contingut mínim
Ordre	Executable i arguments
Procés	PID i PPID
Finalització	Codi de retorn o timeout
Temps	Durada total
Sortides	Línies amb origen i marca temporal

4. Activitats d'anàlisi

A · Processos o fils?

Localitza els noms main, pool-* i io-stdout/io-stderr. Indica quins corresponen a fils i quins elements són processos. Justifica-ho amb PID i PPID.

B · Diagnosi d'errors

Configura un worker perquè acabi amb codi 7. Comprova que el supervisor recull la resta de resultats i que l'error queda explícit.

C · Bloqueig evitat

Explica què podria passar si el supervisor esperés el fill abans de llegir stdout i stderr quan aquest genera molta sortida. Identifica la teva mesura preventiva.

D · Timeout

Crea un worker que superi el límit temporal. Captura la seqüència de finalització i verifica amb ps que no queda viu.

E · Pipe

Fes que el productor generi cinc noms. El consumidor ha de retornar el nombre de línies i el total de caràcters. Dibuixa el recorregut de les dades.

F · Relació amb Fork-Join

Assenyala al teu programa els punts equivalents a fork i join. Explica per què els workers són processos i no tasques d'un ForkJoinPool.

Registre d'incidències

Incidència	Causa	Com s'ha detectat	Solució aplicada

5. Lliurament i avaluació

Evidències obligatòries

- Repositori amb historial coherent.
- README amb requisits, arrencada i proves.
- Captura de stdout i stderr amb PID i PPID.
- Execució completa d'un timeout.
- Pipe productor-consumidor.
- Bateria de proves automatitzades i repetibles.
- Arbre de processos i taula d'identificació completada.
- Informe breu amb decisions, incidències, autoria i ús d'IA.

Criteris d'acceptació

Criteri	Sí/No	Evidència
No usa shell per concatenar entrada externa		
No deixa processos orfes		
Drena stdout i stderr sense interbloqueig		
Errors i timeout tenen resultat explícit		
La bateria de proves és reproduïble		

Preguntes per a la defensa

1. Quina diferència hi ha entre PID, PPID i identificador de fil?
2. Per què ProcessBuilder ha de rebre una llista d'arguments?
3. Per què stdout i stderr s'han de drenar concurrentment?
4. Què aporta tancar stdin després d'escriure-hi?
5. Quina diferència hi ha entre finalització ordenada i forçada?
6. En què s'assembla aquesta solució a Fork-Join i en què se'n diferencia?

CONDICIÓ D'ASSOLIMENT
El producte ha de complir els requisits essencials, disposar de proves reproduïbles i poder ser defensat individualment. Una solució no demostrable o insegura no es considera evidència suficient.

Integració obligatòria · Supervisor com a servei systemd

Pràctica 01 · Supervisor de processos i comunicació IPC

Repte. Configura el supervisor perquè Linux el pugui iniciar i gestionar com un servei systemd. El supervisor executa una feina finita i acaba quan tots els processos fills han finalitzat; configura, per tant, una unitat `Type=oneshot`. Mantén-lo en primer pla: no el converteixis en un dimoni resident ni el facis daemonitzar-se.

Tasques

1. En l'entorn Linux amb systemd autoritzat pel centre, crea la unitat `practica01.service`. Fes servir rutes absolutes i un compte de sistema sense privilegis. No habilitis l'arrencada automàtica de la unitat.

2. Configura el servei amb aquesta base i adapta l'usuari, les rutes i el temps límit al teu entorn:

```
[Unit]
Description=Supervisor multiproces - Practica 01

[Service]
Type=oneshot
User=alumne
WorkingDirectory=/ruta/absoluta/practica01
ExecStart=/usr/bin/java -jar /ruta/absoluta/practica01/supervisor.jar
TimeoutStartSec=90s
KillMode=control-group
StandardOutput=journal
StandardError=journal
```

El límit de systemd ha de ser superior al timeout intern dels workers i deixar temps per tancar-los. `KillMode=control-group` permet que l'aturada del servei s'apliqui als processos que encara queden dins del grup de control.

3. Recarrega la configuració, inicia el servei manualment, comprova'n l'estat i consulta les sortides:

```
sudo systemctl daemon-reload
sudo systemctl start practica01.service
sudo systemctl status practica01.service
sudo journalctl -u practica01.service
```

4. Executa amb systemd una prova correcta i una prova amb timeout o aturada. Com que el supervisor captura stdout/stderr dels workers, reemet al seu propi stdout/stderr les línies que vulguis conservar, identificades amb rol i PID. Verifica al journal els diagnòstics i el resultat, i comprova que no queda cap worker viu després de la cancel·lació. La pipe productor-consumidor s'ha de continuar implementant dins del programa.

Lliurament obligatori

Lliura el fitxer `practica01.service`, les ordres utilitzades i evidències de l'execució gestionada per systemd: estat i codi de sortida, sortides del journal, timeout/aturada i comprovació que no queden processos fills. Explica la funció de `Type=oneshot` i com la unitat gestiona els workers. La validació del servei és obligatòria. Si el teu ordinador no disposa de systemd o no tens permisos per crear una unitat de sistema, fes aquesta part a la màquina virtual o al contenidor de laboratori que proporcioni el centre; no et limitis a descriure la configuració.

Documentació de referència: systemd.service, systemd.exec, systemd.kill i journalctl.